

integration_sec02

Chapter 2: Phase 1 — Foundation and Repository Preparation

Phase 1 establishes the structural and dependency foundation upon which all subsequent integration layers depend. This phase is strictly preparatory: no runtime logic is introduced, no behavioral changes are made to existing translation pipelines, and no provider interactions are altered. Instead, the objective is to create the module references, directory hierarchies, configuration schemas, interface contracts, and governance documentation that enable the incremental integration of LLMsVerifier capabilities across Phases 2–6. The phase boundary is defined by five concrete deliverables: (1) a validated `go.mod` with all transitive dependencies resolved, (2) a complete `internal/verifier/` package tree with stub implementations, (3) an extended configuration schema supporting 33+ environment variables, (4) aligned interface contracts between the existing `llm.LLMClient` abstraction and the LLMsVerifier provider model, and (5) a `CONSTITUTION.md` document codifying 33 mandatory governance rules. All changes in this phase are additive—no existing files are modified except for `go.mod` and `internal/config/config.go`, and those modifications are strictly append-only. The estimated effort for this phase is 16–24 engineering hours, with zero deployment risk because no executable codepaths are activated until the `LLMSVERIFIER_ENABLED` flag is set to `true` in Phase 3.

2.1 Go Module Dependency Management

The first engineering task is to declare LLMsVerifier as a module dependency within the HelixTranslate Go workspace. Because LLMsVerifier is developed as a sibling repository that must be co-located during the integration period, the dependency is established using a combination of a `require` directive and a local `replace` directive. This pattern allows the Go toolchain to resolve the module at build time without requiring a published version to exist on a remote module proxy. The `require` directive pins the semantic version at `v1.0.0`, establishing the interface contract version that subsequent phases assume. The `replace` directive maps this virtual version to a local filesystem path, `./LLMsVerifier`, which is expected to be a Git submodule or a manually cloned sibling directory at the repository root.

The `go.mod` modifications are as follows. First, the `require` block is extended to include the LLMsVerifier module:

```
require (
    digital.vasic.llmsverifier v1.0.0
    // ... existing deps preserved in full
)
```

The replace directive is appended at the end of the go.mod file, after all require blocks:

```
replace digital.vasic.llmsverifier => ./LLMsVerifier
```

These two declarations must appear together; the replace directive without a corresponding require entry will cause the Go compiler to ignore the replacement, while a require without a replace would attempt remote resolution and fail because the module is not yet published to the default module proxy. After adding these lines, execute `go mod tidy` to verify that the Go toolchain can resolve all transitive dependencies introduced by LLMsVerifier.

The addition of LLMsVerifier introduces a significant transitive dependency footprint. Prior to integration, HelixTranslate maintained direct dependencies on nine manually-configured LLM provider SDKs: `github.com/sashabaranov/go-openai` (OpenAI), `github.com/anthropics/anthropic-sdk-go` (Anthropic), `github.com/google/generative-ai-go` (Google Gemini), `cloud.google.com/go/vertexai` (Vertex AI), `github.com/aws/aws-sdk-go-v2/service/bedrockruntime` (Amazon Bedrock), `github.com/Azure/azure-sdk-for-go/sdk/ai/azopenai` (Azure OpenAI), `github.com/deepseek-ai/deepseek-go` (DeepSeek), `github.com/mistralai/client-go` (Mistral AI), and `github.com/cohere-ai/cohere-go/v2` (Cohere). These nine providers were individually imported, initialized, and configured in `internal/translator/providers/`. LLMsVerifier subsumes provider management internally, but the integration strategy preserves these existing imports during the transition period—Phase 4 performs the actual provider consolidation. Therefore, the provider SDKs remain in `go.mod` throughout Phase 1.

Table 2.1: Dependency Audit — Modules Before and After Phase 1 Integration

#	Module Path	Pre-Phase 1	Post-Phase 1	Rationale
1	<code>digital.vasic.llmsverifier</code>	Absent	<code>v1.0.0</code> (local replace)	Core verification engine — required for all scoring, discovery, and selection logic
2	<code>github.com/sashabaranov/go-openai</code>	Present	Present	OpenAI provider SDK; retained during transition, removed in Phase 4
3	<code>github.com/anthropics/anthropic-sdk-go</code>	Present	Present	Anthropic provider SDK; retained during transition
4	<code>github.com/google/generative-ai-go</code>	Present	Present	Google Gemini provider SDK; retained during transition

#	Module Path	Pre-Phase 1	Post-Phase 1	Rationale
5	cloud.google.com/go/vertexai	Present	Present	Vertex AI provider SDK; retained during transition
6	github.com/aws/aws-sdk-go-v2/service/bedrockruntime	Present	Present	Amazon Bedrock provider SDK; retained during transition
7	github.com/Azure/azure-sdk-for-go/sdk/ai/azopenai	Present	Present	Azure OpenAI provider SDK; retained during transition
8	github.com/deepseek-ai/deepseek-go	Present	Present	DeepSeek provider SDK; retained during transition
9	github.com/mistralai/client-go	Present	Present	Mistral AI provider SDK; retained during transition
10	github.com/cohere-ai/cohere-go/v2	Present	Present	Cohere provider SDK; retained during transition
11	github.com/mattn/go-sqlite3	Absent	v1.14.24	LLMsVerifier persistent storage driver; CGO_ENABLED=1 required
12	github.com/gin-gonic/gin	Absent	v1.10.0	LLMsVerifier HTTP API framework; indirect via verifier module
13	github.com/dgrijalva/jwt-go	Absent	v3.2.0+incompatible	LLMsVerifier API authentication middleware
14	golang.org/x/crypto/argon2	Absent	v0.36.0	LLMsVerifier credential hashing for admin API endpoints
15	github.com/stretchr/testify	Absent	v1.10.0	Test assertions for verifier unit tests; indirect via verifier module
16	github.com/testcontainers/testcontainers-go	Absent	v0.35.0	Integration test infrastructure for SQLite-backed verifier scenarios
17	golang.org/x/sync/errgroup	Absent	v0.12.0	Concurrent goroutine coordination in

#	Module Path	Pre-Phase 1	Post-Phase 1	Rationale
18	google.golang.org/grpc	v1.76.0	v1.77.0	multi-provider discovery Version conflict resolution: LLMsVerifier requires v1.77.0 for new health-check streaming API

Table 2.1 enumerates 18 module entries, of which 13 represent additions or version changes relative to the pre-integration baseline. The most operationally significant change is the `google.golang.org/grpc` version conflict: HelixTranslate previously pinned `grpc` at `v1.76.0` for compatibility with a legacy telemetry exporter, while LLMsVerifier requires `v1.77.0` for its streaming health-check API. The resolution is to upgrade HelixTranslate's `grpc` dependency to `v1.77.0` and update the telemetry exporter initialization code (a single-line change in `internal/telemetry/exporter.go`) to use the new `grpc.DialContext` signature. This change is made in Phase 1 because it is a prerequisite for `go mod tidy` to resolve successfully.

Go Version Alignment. At the start of Phase 1, HelixTranslate targets Go 1.25.2 and HelixAgent targets Go 1.26. LLMsVerifier requires Go 1.25.3 as its minimum version because it uses the `iter.Seq` generic iterator pattern introduced in that patch release for its model discovery streaming API. The decision is to standardize all three modules on Go 1.25.3:

go 1.25.3

This is the minimum version that satisfies all three modules. HelixAgent's `go 1.26` directive is downgraded to `1.25.3`; this is safe because HelixAgent does not use any Go 1.26-specific language features. The `toolchain` directive is set to `go1.25.3` to ensure consistent compiler behavior across development environments and CI pipelines. After changing the version directive, verify with `go version` and `go vet ./...` across all three module roots.

The `github.com/mattn/go-sqlite3` dependency (row 11) requires special attention because it uses CGO to bind the SQLite C library. This means that all build environments—including CI runners, Docker build stages, and developer workstations—must have `CGO_ENABLED=1` set. The existing HelixTranslate build used `CGO_ENABLED=0` for static binary production. Phase 1 introduces a dual-build strategy: development and test builds use `CGO_ENABLED=1` to support SQLite-backed verifier storage, while production release builds continue to use `CGO_ENABLED=0` until Phase 5 completes the migration to a non-CGO persistence backend. This is documented in `Makefile` and `Dockerfile` comments during Phase 1.

2.2 Project Structure Creation

With module dependencies declared, the next deliverable is the directory hierarchy for the `internal/verifier/` package and its subpackages. All directories are created as empty shells with `.gitkeep` files, with the exception of `configs/verifier.yaml` which is populated with a template configuration. The structure is designed to mirror the five functional domains of LLMsVerifier—scoring, discovery, verification, selection, and configuration—while remaining idiomatic to Go package organization conventions.

Create the following directories using `mkdir -p`:

- `internal/verifier/scoring/` — Weighted scoring engine and component calculators
- `internal/verifier/discovery/` — Three-tier model discovery service and registry
- `internal/verifier/verification/` — Verification pipeline and adapter layer
- `internal/verifier/selection/` — Model selection engine with fallback chain
- `internal/verifier/challenges/` — Executable challenge scripts for subsystem validation
- `internal/verifier/config/` — Verifier-specific configuration types and validation
- `internal/services/` — Top-level service composition (shared with existing HelixTranslate services)

Each subdirectory contains a `doc.go` file with a package-level comment explaining its responsibility. The root `internal/verifier/` directory holds six files that serve as the public API surface for the package: `client.go` (HTTP client for LLMsVerifier API communication), `health.go` (health check integration with the existing `/healthz` endpoint), `pipeline.go` (the 8-step verification pipeline coordinator), `config.go` (configuration struct definitions), `bridge.go` (interface bridge between HelixTranslate's `llm.LLMClient` and LLMsVerifier's provider model), and `errors.go` (verifier-specific error types). The `challenges/` directory is unique in that it contains shell scripts rather than Go source files—these are executable validation scenarios that engineers run manually or in CI to verify subsystem behavior.

The `configs/verifier.yaml` template is created at the repository root to provide a reference configuration that operators can copy and customize:

```
verifier:
  enabled: true
  api_url: http://localhost:8080
  db_path: ./data/verifier.db
  cache_ttl: 1h
  providers:
    - id: openai
      api_key: ${OPENAI_API_KEY}
      models: []
    - id: anthropic
      api_key: ${ANTHROPIC_API_KEY}
      models: []
```

```

scoring:
  weights:
    response_speed: 0.20
    cost_effectiveness: 0.30
    model_efficiency: 0.25
    capability: 0.20
    recency: 0.05
discovery:
  mode: auto
  refresh_interval: 1h

```

This YAML template encodes the default configuration values that match the tagged struct defaults in `internal/config/config.go` (Section 2.3). The providers section demonstrates the expected structure for provider entries: each provider has an opaque string `id`, an `api_key` that is resolved from environment variables using `${VAR}` interpolation, and a `models` array that starts empty because the discovery service populates it automatically at runtime. The `scoring.weights` subsection defines the five-component weight vector that the composite scoring engine uses; these values sum to `1.0` and can be adjusted per-environment to prioritize different operational objectives. The `discovery.mode: auto` setting enables the three-tier discovery pipeline (registry scan → health validation → capability assessment), while `refresh_interval: 1h` controls how frequently the discovery service invalidates its cached model catalog.

The complete directory tree for the verifier package is as follows:

```

internal/verifier/
├── client.go           # HTTP client for LLMsVerifier
├── health.go           # Health check integration
├── pipeline.go         # 8-step verification pipeline
├── config.go           # Verifier configuration structs
├── bridge.go           # Interface bridge between abstractions
├── errors.go           # Verifier-specific error types
├── scoring/
│   ├── doc.go          # Package documentation
│   ├── engine.go       # Scoring engine wrapper
│   ├── components.go   # 5 component score calculators
│   ├── composite.go    # Weighted score aggregator
│   └── history.go      # Score persistence and time-series
├── discovery/
│   ├── doc.go          # Package documentation
│   ├── service.go      # 3-tier discovery coordinator
│   └── registry.go     # Model registry (SSOT for available
models)
├── gatekeeper.go       # Model availability gate
├── sync.go             # Background synchronization worker
├── selection/
│   ├── doc.go          # Package documentation
│   ├── engine.go       # Model selection logic
│   └── fallback.go     # Fallback chain and retry logic
├── challenges/
│   └── score_adapter.sh # Challenge: adapter accuracy
validation
├── gatekeeping.sh      # Challenge: gatekeeper threshold

```

```

enforcement
  └─ discovery.sh      # Challenge: discovery resilience under
failure
  └─ selection.sh      # Challenge: selection fairness across
providers

```

The `client.go` file at the root of `internal/verifier/` implements an HTTP client that communicates with the LLMsVerifier service over its REST API. It is responsible for connection pooling, request signing (using the `LLMSVERIFIER_API_KEY` environment variable), response deserialization, and circuit-breaker logic. The `health.go` file registers a verifier health probe with the existing `internal/health` package, adding a `/healthz/verifier` endpoint that reports the connectivity status to the LLMsVerifier API and the SQLite database. The `pipeline.go` file defines the 8-step verification pipeline that is fully implemented in Phase 3; in Phase 1 it contains only the pipeline struct definition and a no-op `Run()` method that returns an unimplemented error. The `bridge.go` file defines the `VerifiedProvider` interface (detailed in Section 2.4) and its adapter implementations.

The `challenges/` subdirectory deserves special mention. Unlike the Go source files in other directories, the four files here are executable Bash scripts that serve as subsystem-level validation scenarios. Each script performs a specific challenge: `score_adapter.sh` validates that the scoring engine correctly maps raw performance metrics to normalized component scores; `gatekeeping.sh` verifies that the gatekeeper correctly rejects unavailable models before they reach the selection engine; `discovery.sh` tests discovery resilience by simulating provider API failures and verifying graceful degradation; `selection.sh` confirms that the selection engine distributes load fairly across providers with equivalent scores. These scripts are executed manually during development and automatically in CI after the full integration is complete. They are not Go tests because they operate at the integration level and require external service dependencies.

2.3 Configuration Schema Design

HelixTranslate uses a unified configuration system in which all application settings are defined as Go structs with struct tags for JSON deserialization, environment variable binding, and default value specification. Phase 1 extends this system by adding a new `LLMsVerifierConfig` struct to `internal/config/config.go` and registering it within the existing `AppConfig` root struct.

The complete configuration struct definition is:

```

type LLMsVerifierConfig struct {
    Enabled      bool           `json:"enabled"
                        env:"LLMSVERIFIER_ENABLED" default:"true"`
    APIURL       string         `json:"api_url"
                        env:"LLMSVERIFIER_API_URL" default:"http://
                        localhost:8080"`
    APIKey       string         `json:"api_key"
                        env:"LLMSVERIFIER_API_KEY" required:"true"`
    DBPath       string         `json:"db_path"
                        env:"LLMSVERIFIER_DB_PATH" default:"./data/verifier.db"`
}

```

```

CacheTTL    time.Duration    `json:"cache_ttl"
           env:"LLMSVERIFIER_CACHE_TTL" default:"1h"`
Providers   []ProviderConfig `json:"providers"`
Scoring     ScoringConfig   `json:"scoring"`
Discovery   DiscoveryConfig  `json:"discovery"`
}

```

This struct is embedded into the root `AppConfig` struct as a field named `Verifier` of type `*LLMsVerifierConfig`. Using a pointer type allows the configuration loader to distinguish between "verifier configuration explicitly disabled" (`Verifier: nil` after parsing when the `verifier` key is absent from YAML) and "verifier configuration present with default values" (a non-nil struct when the key is present). The three substructs—`ProviderConfig`, `ScoringConfig`, and `DiscoveryConfig`—are defined in the same file. `ProviderConfig` captures per-provider settings including API key, model allowlists, rate limits, and custom headers. `ScoringConfig` holds the five weight values and normalization parameters. `DiscoveryConfig` controls the discovery mode (auto, manual, or hybrid), refresh intervals, timeout values, and concurrency limits.

The configuration system supports three override layers in priority order: (1) environment variables, (2) YAML configuration files, (3) struct tag defaults. The `env` tags on the struct fields define the environment variable names that the loader inspects. The `required:"true"` tag on `APIKey` causes the application to exit with a fatal error during startup if the key is not provided via environment variable or configuration file. This is the only required field; all other fields have sensible defaults that allow the verifier subsystem to start in a local development configuration without any explicit settings.

Table 2.2: Complete Environment Variable Registry for LLMsVerifier Subsystem

#	Environment Variable	Type	Default	Required
1	LLMSVERIFIER_ENABLED	bool	true	No
2	LLMSVERIFIER_API_URL	string	http://localhost:8080	No
3	LLMSVERIFIER_API_KEY	string	—	Yes
4	LLMSVERIFIER_DB_PATH	string	./data/verifier.db	No

#	Environment Variable	Type	Default	Required
5	LLMSVERIFIER_CACHE_TTL	duration	1h	No
6	LLMSVERIFIER_VERIFICATION_ENABLED	bool	true	No
7	LLMSVERIFIER_SCORING_ENABLED	bool	true	No
8	LLMSVERIFIER_DISCOVERY_ENABLED	bool	true	No
9	LLMSVERIFIER_MAX_CONCURRENT	int	10	No
10	LLMSVERIFIER_TIMEOUT	duration	30s	No
11	LLMSVERIFIER_WEIGHT_SPEED	float64	0.20	No
12	LLMSVERIFIER_WEIGHT_COST	float64	0.30	No
13	LLMSVERIFIER_WEIGHT EFFICIENCY	float64	0.25	No
14	LLMSVERIFIER_WEIGHT_CAPABILITY	float64	0.20	No
15	LLMSVERIFIER_WEIGHT_RECENCY	float64	0.05	No

#	Environment Variable	Type	Default	Required
16	LLMSVERIFIER_DISCOVERY_MODE	string	auto	No
17	LLMSVERIFIER_DISCOVERY_REFRESH_INTERVAL	duration	1h	No
18	LLMSVERIFIER_DISCOVERY_TIMEOUT	duration	10s	No
19	LLMSVERIFIER_GATEKEEPER_STRICT	bool	false	No
20	LLMSVERIFIER_FALLBACK_CHAIN	string	score,tier,cost	No
21	LLMSVERIFIER_LOG_LEVEL	string	info	No
22	OPENAI_API_KEY	string	—	No*
23	ANTHROPIC_API_KEY	string	—	No*
24	GEMINI_API_KEY	string	—	No*
25	VERTEX_AI_PROJECT_ID	string	—	No*

#	Environment Variable	Type	Default	Required
26	VERTEX_AI_LOCATION	string	us-central1	No*
27	AWS_REGION	string	—	No*
28	AWS_ACCESS_KEY_ID	string	—	No*
29	AWS_SECRET_ACCESS_KEY	string	—	No*
30	AZURE_OPENAI_ENDPOINT	string	—	No*
31	AZURE_OPENAI_API_KEY	string	—	No*
32	DEEPSEEK_API_KEY	string	—	No*
33	MISTRAL_API_KEY	string	—	No*
34	COHERE_API_KEY	string	—	No*
35	LLMSVERIFIER_METRICS_ENABLED	bool	true	No
36	LLMSVERIFIER_METRICS_PORT	int	9090	No

Table 2.2 documents 36 environment variables (exceeding the 33+ requirement). Variables 1–21 control the verifier subsystem itself; variables 22–34 are the existing provider API keys that were already present in HelixTranslate's configuration but are now explicitly enumerated because the verifier's discovery service consumes them during the registry population phase. Variables 35–36 are new additions for observability integration. The five scoring weights (variables 11–15) are validated at startup to ensure they sum to exactly 1.0; if they do not, the application logs a warning and normalizes them proportionally.

The LLMSVERIFIER_API_KEY (variable 3) is the only field marked required:"true" at the verifier configuration level. This key is a JWT bearer token generated by the LLMsVerifier administrator panel during initial deployment.

It is used by the `client.go` HTTP client to sign all outgoing requests with an `Authorization: Bearer <token>` header. The token contains claims for rate-limiting identity and access scope; the verifier API validates the signature against a pre-shared secret. Rotation is supported via a `LLMSVERIFIER_API_KEY_PREVIOUS` environment variable that accepts the previous token for a grace period of 24 hours, preventing authentication failures during key rotation events.

Weight configuration deserves careful treatment because the five scoring components are interdependent. The default weights of `0.20/0.30/0.25/0.20/0.05` prioritize cost-effectiveness (30%) in a production translation workload where API spend is the dominant operational concern, followed by model efficiency (25%) which captures throughput per dollar. Response speed and capability each receive 20%, reflecting equal importance between latency and quality. Recency receives the smallest weight (5%) as a tiebreaker rather than a primary discriminator. Operators can adjust these weights per-environment: a development environment might set `LLMSVERIFIER_WEIGHT_SPEED=0.40` to prioritize fast iteration, while a high-stakes production deployment might set `LLMSVERIFIER_WEIGHT_CAPABILITY=0.40` to prioritize translation quality above all else.

2.4 Interface Alignment

HelixTranslate's existing LLM abstraction is defined in `pkg/translator/llm/provider.go` as the `LLMClient` interface. `LLMsVerifier` defines a similar but not identical abstraction in its own package. Phase 1 must align these two interfaces so that HelixTranslate's existing provider implementations can be wrapped by the verifier's scoring and selection layers without requiring changes to the caller code. This alignment is achieved through a bridge pattern implemented in `internal/verifier/bridge.go`.

The existing `LLMClient` interface in `pkg/translator/llm/provider.go` is:

```
type LLMClient interface {
    Translate(ctx context.Context, text string, sourceLang,
        targetLang string) (string, error)
    GetModelInfo(ctx context.Context) (ModelInfo, error)
    HealthCheck(ctx context.Context) error
    Close() error
}
```

`LLMsVerifier`'s corresponding provider interface (defined in the `LLMsVerifier` module) is:

```
type Provider interface {
    Completion(ctx context.Context, prompt string, params
        CompletionParams) (CompletionResponse, error)
    GetModelMetadata(ctx context.Context) (ModelMetadata, error)
    Ping(ctx context.Context) error
    Close() error
}
```

Table 2.3: Method Mapping Between HelixTranslate LLMClient and LLMsVerifier Provider

HelixTranslate Method	LLMsVerifier Method	Semantic Equivalence	Adapter Logic
Translate(ctx, text, srcLang, tgtLang)	Completion(ctx, prompt, params)	Core inference: both execute a prompt against a language model	Convert translation request to prompt template; map source/target languages to params.Languages; extract text from CompletionResponse.Text
GetModelInfo(ctx)	GetModelMetadata(ctx)	Metadata retrieval: both return model identification and capability information	Map ModelInfo (name, version, maxTokens) to ModelMetadata (name, version, contextWindow, capabilities, capability bitmap translation)
HealthCheck(ctx)	Ping(ctx)	Liveness probe: both verify the provider endpoint is reachable and responsive	Direct pass-through with error translation; Ping() returns nil — HealthCheck() returns nil; Ping() returns error → wrap in verifier.ErrProviderUnavailable
Close()	Close()	Resource cleanup: both release connections, close HTTP clients, and free handles	Direct delegation; no transformation needed

Table 2.3 reveals that the two interfaces share identical semantic intent but differ in naming conventions and parameter structures. The `Close()` method is a direct match requiring no adapter logic. The health-check methods differ only in name (`HealthCheck` vs. `Ping`) and can be bridged with a simple wrapper. The metadata methods differ in return types (`ModelInfo` vs. `ModelMetadata`) but both convey the same essential information: model identity, version, and operational parameters. The translation and completion methods differ most significantly: `Translate` takes structured language parameters while `Completion` takes a raw prompt string and parameter bag. The adapter must perform prompt templating—converting the `(text, sourceLang, targetLang)` triple into a translation prompt such as `"Translate the following text from {sourceLang} to {targetLang}: \n\n{text}"`—and extract the translated text from the completion response.

The bridge interface that unifies both abstractions is defined in `internal/verifier/bridge.go`:

```

type VerifiedProvider interface {
    llm.LLMClient
    GetModelID() string
    GetVerificationStatus() VerificationStatus
    GetCompositeScore() *CompositeScore
    GetTier() ModelTier
    GetCapabilities() CapabilityBitmap
}

```

The `VerifiedProvider` interface embeds `llm.LLMClient`, meaning it inherits all four methods (`Translate`, `GetModelInfo`, `HealthCheck`, `Close`) and is substitutable wherever an `LLMClient` is expected. This is a critical design decision: because `VerifiedProvider` is a subtype of `LLMClient`, the existing `Translator` struct in `internal/translator/engine.go` can accept `VerifiedProvider` instances without modification. The additional methods—`GetModelID`, `GetVerificationStatus`, `GetCompositeScore`, `GetTier`, and `GetCapabilities`—are provided by the verifier subsystem to enable scoring-aware selection while preserving backward compatibility.

The `VerificationStatus` type is an enumerated string with values `StatusUnknown`, `StatusPending`, `StatusVerified`, `StatusDegraded`, and `StatusFailed`. A provider in `StatusVerified` has passed all 8 verification steps (Phase 3) within the last cache TTL period. `StatusDegraded` indicates that the provider is functional but has non-critical failures in some verification steps (e.g., elevated latency within acceptable bounds). `StatusFailed` means the provider has critical failures and should not receive traffic. The gatekeeper (Section 2.2, `gatekeeper.go`) uses this status to filter providers before they reach the selection engine.

The `CompositeScore` struct aggregates the five component scores into a single normalized value between 0.0 and 1.0, along with the individual component breakdowns for observability. The `ModelTier` type classifies models into `TierFree`, `TierStandard`, `TierPremium`, and `TierEnterprise` based on their pricing tier and capability profile. The `CapabilityBitmap` is a 64-bit bitmask where each bit represents a specific capability (e.g., multilingual translation, code generation, long-context processing, tool use, vision understanding); this enables fast capability matching during model selection.

The bridge implementation includes two concrete adapter structs: `VerifiedProviderAdapter` wraps an existing `llm.LLMClient` implementation (such as `OpenAIProvider` or `AnthropicProvider`) and adds the verifier-specific methods by delegating to the `LLMsVerifier` API, and `NativeVerifiedProvider` wraps an `LLMsVerifier` Provider directly and implements the `llm.LLMClient` interface. Both adapters implement the full `VerifiedProvider` interface, allowing the selection engine to work with either native `LLMsVerifier` providers or legacy `HelixTranslate` providers that have been enrolled in the verification system.

2.5 Constitution and Governance Documentation

The final deliverable of Phase 1 is a governance document that codifies the rules, invariants, and constraints governing the `LLMsVerifier` integration. This document, `internal/verifier/CONSTITUTION.md`, serves as the authoritative reference for

all engineering decisions affecting the verifier subsystem. It is not a design document—it is a normative specification that defines what the system must, must not, and should do. Each rule is assigned a unique identifier for traceability in code comments, commit messages, and issue tracking.

The constitution contains 33 mandatory rules organized into six categories. The document begins with a preamble establishing its authority and scope, followed by the rule definitions, followed by an appendix describing the amendment process.

Table 2.4: Constitution Rule Mapping by Category

Category	Rule IDs	Count	Coverage Area
Scoring	S01–S08	8	Weight normalization, score calculation, component aggregation, score persistence, caching, recency decay, outlier rejection, score transparency
Verification	V01–V08	8	8-step pipeline execution, adapter accuracy, timeout handling, error classification, retry policy, health check frequency, result caching, verification audit trail
Discovery	D01–D05	5	Registry consistency, provider enumeration, model metadata validation, discovery refresh scheduling, graceful degradation on provider failure
Selection	L01–L05	5	Score-based ranking, fallback chain execution, tier-based filtering, capability matching, load distribution fairness
Testing	T01–T04	4	Unit test coverage thresholds, integration test requirements, challenge script validation, performance regression detection
Operations	O01–O03	3	Deployment rollback criteria, monitoring and alerting, incident response procedures

Table 2.4 maps the 33 rules across six functional categories. Each rule is written as a normative statement using RFC 2119 keywords (MUST, MUST NOT, SHOULD, MAY). The scoring rules (S01–S08) govern the five-component scoring engine: S01 mandates that weights always sum to 1.0; S02 requires score normalization to the [0, 1] interval; S03 defines the recency decay function (exponential half-life of 24 hours); S04 mandates outlier rejection using the interquartile range method; S05 requires score persistence to SQLite within 5 seconds of calculation; S06 mandates a 1-hour TTL for cached scores; S07 requires score transparency (all component scores must be queryable via the API); S08 mandates that score changes greater than 0.1 trigger an audit log entry.

The verification rules (V01–V08) govern the 8-step pipeline: V01 mandates that all 8 steps execute in sequence and that partial results are discarded if any step fails critically; V02 requires that adapter accuracy (the fidelity of the bridge between LLMClient and Provider) be validated on every startup with a known-good translation pair; V03 defines a 30-second global timeout per verification request with per-step budget allocation; V04 mandates error classification into Critical, Degraded, and Transient categories; V05 requires exponential backoff with jitter

for retry attempts (base 100ms, max 30s); V06 mandates health checks every 60 seconds for active providers; V07 requires verification results to be cached with the same TTL as scores; V08 mandates an audit trail recording every verification event with timestamp, provider ID, step results, and outcome.

The discovery rules (D01–D05) govern the 3-tier discovery service: D01 mandates that the registry be the single source of truth (SSOT) for all model availability data; D02 requires enumeration of all configured providers on startup and at every refresh interval; D03 mandates that discovered model metadata be validated against a schema (ID format, version string, capability bitmap range); D04 requires scheduled refresh with jitter ($\pm 10\%$ of the interval) to prevent thundering herd; D05 mandates graceful degradation—if a provider fails discovery, its previously cached models remain available with a stale flag until the next successful discovery or explicit timeout.

The selection rules (L01–L05) govern model selection: L01 mandates score-based ranking as the primary selection criterion; L02 requires the fallback chain (score, tier, cost by default) to execute in order when the primary criterion yields ties; L03 mandates tier-based filtering when the request specifies a minimum tier; L04 requires capability matching—models without the requested capabilities must be excluded before scoring; L05 mandates load distribution fairness: among providers with equivalent composite scores (difference < 0.01), selection must distribute requests with uniform probability.

The testing rules (T01–T04) establish quality gates: T01 mandates 85% unit test coverage for all `internal/verifier/` packages; T02 requires integration tests for each of the four challenge scripts; T03 mandates that all challenge scripts pass in CI before any pull request affecting the verifier subsystem can be merged; T04 requires performance regression detection—any change that increases the p99 verification latency by more than 10% must be flagged in CI and reviewed.

The operations rules (O01–O03) cover deployment and runtime: O01 mandates automatic rollback if the verifier health check fails for more than 2 minutes post-deployment; O02 requires Prometheus metrics for all verifier operations with defined alert thresholds; O03 mandates an incident response runbook for verifier-related outages, including a decision tree for disabling the verifier via `LLMSVERIFIER_ENABLED=false` as an emergency mitigation.

Two additional documents are updated during Phase 1. `CLAUDE.md` at the repository root receives a new section describing the verifier subsystem architecture and its interaction points with the translation engine. `AGENTS.md` (the agent configuration file for AI-assisted development) receives updated context about the `internal/verifier/` package structure, the location of key interfaces, and the rule identifiers from the constitution that should be referenced when generating code. These updates ensure that AI-assisted development tools have accurate context for the new subsystem.

The constitution is a living document governed by an amendment process: any change to a rule requires (a) a pull request with rationale, (b) review by at least two subsystem owners, (c) update of the rule's version timestamp, and (d) propagation of the change to all code comments referencing the rule ID. Rules marked with `[IMMUTABLE]` in the constitution—specifically S01 (weight normalization), D01 (registry SSOT), and L05 (fairness)—cannot be amended without a formal architectural review and sign-off from the technical lead.

Phase 1 concludes when all five deliverables are in place: the `go.mod` resolves without errors and all 18 dependency entries are validated; the `internal/verifier/` directory tree is created with stub files and documented package structure; the `LLMsVerifierConfig` struct is defined and the configuration loader recognizes all 36 environment variables; the `VerifiedProvider` bridge interface is defined with both adapter struct types; and the `CONSTITUTION.md` document is created with all 33 rules mapped and traceable. At this point, the codebase compiles successfully with `go build ./...`, all existing tests pass, and no verifier codepaths are active in production. The foundation is ready for Phase 2: Integration and Verification Pipeline Development.